
Data Structures

BS (CS) _Fall_2025

Lab_02 Task



Learning Objectives:

1. Class Templates with unit testing (Gtest)

Class Templates in C++

Task 0: Simple Generic Holder

Create a template class `Holder<T>` that stores a single item of any type.

Data Members:

- **T item:** the stored value.

Functions:

- **Constructor:** initialize the item.
- **setItem(T value):** set the stored item.
- **getItem():** return the stored item.
- **displayType():** prints the data type of the stored item using `typeid(item).name()` from `typeinfo` header.

Test Cases:

- Create `Holder<int>`, `Holder<double>`, and `Holder<string>`.
- Set and get values for each.
- Print stored type and value.

Task 1:

Define a template class **Container** for storing different type of values e.g. int, double, float etc. This container would keep same type of values/elements at a time. The **Container** class should consist of the following member variables:

T * values; A pointer to set of values it contains. This member points to the dynamically allocated array (which you will be allocating in constructor).

capacity; An integer that shows total capacity of the container

counter; An integer counter variable which increments upon every insertion operation and decrements upon removal of any element; representing total number of elements currently present in the container.

The container should consist of following functions:

- **constructor** with capacity as parameter
- **isFull** which will return true if counter reaches capacity otherwise false
- **insert** which will receive a value of some type (int/double/float) and insert into the container if it's not already full

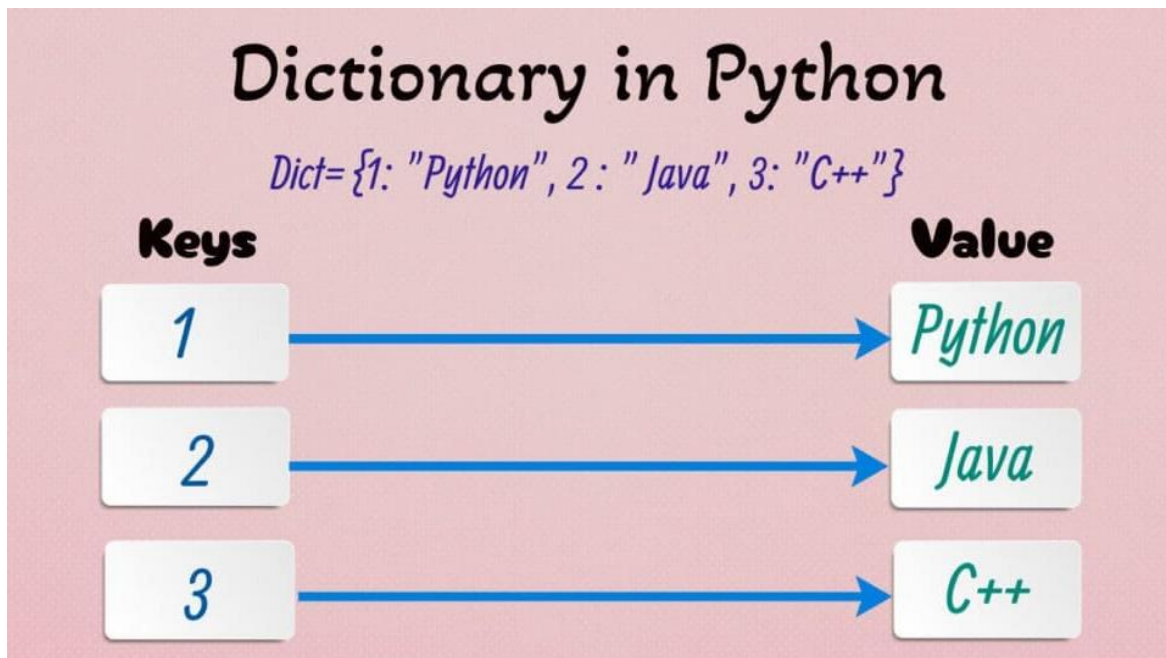
- **search** which will return true if the container contains value passed as parameter
- **remove** which will remove a value, if it exists, from the container and shifts all subsequent values one index back.
- **print** which will print all values contained in the container

Now create a test class to create three instances of Container for types: **int**, **float**, **double** and call all functions separately for each instance. Your output must be properly formatted.

Task 2:

Template Class Dictionary:

You are designing a generic Dictionary class similar to Python's dict that stores key–value pairs of any type.



The dictionary should be flexible enough to:

- Work with any combination of key and value types.
- Allow capacity to be set at runtime when the object is created.
- Provide default types for convenience.
- Have a special behavior when keys are std::string → make it case-insensitive by automatically converting keys to lowercase and performing searches/removals ignoring case.

The class will have Three template parameters:

- K for the key type (default: int)
- V for the value type (default: string)

Data Members (Private)

- **K* keys;** :- dynamic array storing keys.
- **V* values;** :- dynamic array storing corresponding values.
- **int count;** :- current number of elements.
- **int currentSize;** :- current allocated size of arrays (starts small, doubles when needed).

Member Functions (Public)

Constructor

- Initializes count = 0, currentSize = 2.
- Allocates keys = new K[currentSize] and values = new V[currentSize].

Destructor

- Deletes dynamically allocated arrays.

isEmpty()

- Returns true if count == 0.

insert(K key, V value)

- If the key already exists, do nothing.
- If count == currentSize, reallocate arrays with double the size.
- Insert the new key–value pair.

search(K key)

- Returns true if key exists, otherwise false.

remove(K key)

- If found, remove the key–value pair and shift the rest of the elements.

getValue(K key)

- Returns the value if key exists, otherwise throws an exception.

print()

- Prints all key–value pairs.

Test cases :

1. General Dictionary Tests (Default Parameters)(K = int, V = std::string)

- Create Dictionary<>.
- Insert 3+ key value pairs:
 - Example: 1 : "Ali", 2 : "Sara", 3 : "John". Insert duplicate key (e.g., 2 : "Sara") :- should not add.
- Search existing key (2) :- returns true.
- Search non-existing key (10) :- returns false.
- Get value of existing key (2) :- returns "Sara".
- Remove key (2) :- confirm it's gone.
- Print dictionary when empty and when not empty.

2. Auto resizing Tests : (K = int, V = double)

- Create Dictionary<int, double>.
- Insert more elements than initial size (e.g., > 2) with values like 101 : 90.5, 102 : 85.3, etc.
- Verify resizing happens and all inserted elements remain accessible.

3. String Key Specialization Tests (Case Insensitive)(K = std::string, V = int)

- Create Dictionary<std::string, int>.
- Insert "Apple" : 5, "Banana" : 10.
- Search "APPLE", "apple", "ApPIE" : all return true.
- Remove "APPLE" , confirm "apple" is removed.

4. Different Data Type Combination Tests

i) Double keys & char values

- Create Dictionary<double, char>.
- Insert 1.1 : 'A', 2.2 : 'B'.
- Search 2.2 , returns true.
- Remove 1.1 , confirm removal.

ii) Char keys & bool values

- Create Dictionary<char, bool>.
- Insert 'x' : true, 'y' : false.
- Search 'y' , returns true.
- Get value 'x' , returns true.